digits

Grupprojekt

DATASET:SCIKIT-LEARN DIGITS

1797 BILDER, 8*8 PIXLAR, INT 0-16

DATAREDUKTION

Nicklas Thegerström

Emil Sjökvist

Fredrik Käll

Datareduktion

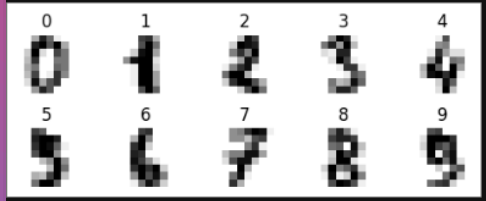
- Ladda data, inspektera, kör lite statistik för att börja bekanta oss med datasetet.

```python
digits = load_digits()

# DF i utforskningssyfte
df = pd.DataFrame(np.column_stack([digits['data'],digits['target']]), columns=digits['feature_names']+['target'])

print("df.info")
print(df.info())
print("df.describe")
print(df.describe())
print("df.head")
print(df.head())

# x och y är var vår arbetsdata är
x = digits['data']
y = digits['target']
images = digits['images']

# se vad vi har i x och y
print(x.shape, y.shape)

# Gör en subplot för att visualisera datan
fig, axes = plt.subplots(2, 5, figsize=(5, 2))

for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='gray_r')
    ax.set_title(y[i])
    ax.axis('off')

plt.tight_layout()
plt.show()

# Hur många av varje siffra har vi?
counts = np.bincount(y)
for digit, count in enumerate(counts):
    print(f"Siffra {digit}: {count}")
```

```
df.info
<class 'pandas.DataFrame'>
RangeIndex: 1797 entries, 0 to 1796
Data columns (total 65 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   pixel_0_0   1797 non-null   float64
 1   pixel_0_1   1797 non-null   float64
 2   pixel_0_2   1797 non-null   float64
 3   pixel_0_3   1797 non-null   float64
 4   pixel_0_4   1797 non-null   float64
 5   pixel_0_5   1797 non-null   float64
 6   pixel_0_6   1797 non-null   float64
 7   pixel_0_7   1797 non-null   float64
 8   pixel_1_0   1797 non-null   float64
 9   pixel_1_1   1797 non-null   float64
 10  pixel_1_2   1797 non-null   float64
 11  pixel_1_3   1797 non-null   float64
 12  pixel_1_4   1797 non-null   float64
 13  pixel_1_5   1797 non-null   float64
 14  pixel_1_6   1797 non-null   float64
 15  pixel_1_7   1797 non-null   float64
 16  pixel_2_0   1797 non-null   float64
 17  pixel_2_1   1797 non-null   float64
 18  pixel_2_2   1797 non-null   float64
 19  pixel_2_3   1797 non-null   float64
 20  pixel_2_4   1797 non-null   float64
 21  pixel_2_5   1797 non-null   float64
 22  pixel_2_6   1797 non-null   float64
 23  pixel_2_7   1797 non-null   float64
 24  pixel_3_0   1797 non-null   float64
 25  pixel_3_1   1797 non-null   float64
 26  pixel_3_2   1797 non-null   float64
 27  pixel_3_3   1797 non-null   float64
 28  pixel_3_4   1797 non-null   float64
 29  pixel_3_5   1797 non-null   float64
 30  pixel_3_6   1797 non-null   float64
 31  pixel_3_7   1797 non-null   float64
 32  pixel_4_0   1797 non-null   float64
 33  pixel_4_1   1797 non-null   float64
 34  pixel_4_2   1797 non-null   float64
 35  pixel_4_3   1797 non-null   float64
 36  pixel_4_4   1797 non-null   float64
 37  pixel_4_5   1797 non-null   float64
 38  pixel_4_6   1797 non-null   float64
 39  pixel_4_7   1797 non-null   float64
 40  pixel_5_0   1797 non-null   float64
 41  pixel_5_1   1797 non-null   float64
 42  pixel_5_2   1797 non-null   float64
 43  pixel_5_3   1797 non-null   float64
 44  pixel_5_4   1797 non-null   float64
 45  pixel_5_5   1797 non-null   float64
 46  pixel_5_6   1797 non-null   float64
 47  pixel_5_7   1797 non-null   float64
 48  pixel_6_0   1797 non-null   float64
 49  pixel_6_1   1797 non-null   float64
 50  pixel_6_2   1797 non-null   float64
 51  pixel_6_3   1797 non-null   float64
 52  pixel_6_4   1797 non-null   float64
 53  pixel_6_5   1797 non-null   float64
 54  pixel_6_6   1797 non-null   float64
 55  pixel_6_7   1797 non-null   float64
 56  pixel_7_0   1797 non-null   float64
 57  pixel_7_1   1797 non-null   float64
 58  pixel_7_2   1797 non-null   float64
 59  pixel_7_3   1797 non-null   float64
 60  pixel_7_4   1797 non-null   float64
 61  pixel_7_5   1797 non-null   float64
 62  pixel_7_6   1797 non-null   float64
 63  pixel_7_7   1797 non-null   float64
 64  target      1797 non-null   float64
dtypes: float64(65)
memory usage: 912.7 KB
```

```
df.describe
       pixel_0_0   pixel_0_1   pixel_0_2   pixel_0_3   pixel_0_4  \
count     1797.0  1797.000000  1797.000000  1797.000000  1797.000000
mean         0.0     0.303840     5.204786    11.835838    11.848080
std          0.0     0.907192     4.754826     4.248842     4.287388
min          0.0     0.000000     0.000000     0.000000     0.000000
25%          0.0     0.000000     1.000000    10.000000    10.000000
50%          0.0     0.000000     4.000000    13.000000    13.000000
75%          0.0     0.000000     9.000000    15.000000    15.000000
max          0.0     8.000000    16.000000    16.000000    16.000000

       pixel_0_5   pixel_0_6   pixel_0_7   pixel_1_0   pixel_1_1  ... \
count  1797.000000  1797.000000  1797.000000  1797.000000  1797.000000  ...
mean      5.781859     1.362270     0.129661     0.005565     1.993879  ...
std       5.666418     3.325775     1.037383     0.094222     3.196160  ...
min       0.000000     0.000000     0.000000     0.000000     0.000000  ...
25%       0.000000     0.000000     0.000000     0.000000     0.000000  ...
50%       4.000000     0.000000     0.000000     0.000000     0.000000  ...
75%      11.000000     0.000000     0.000000     0.000000     3.000000  ...
max      16.000000    16.000000    15.000000     2.000000    16.000000  ...

       pixel_6_7   pixel_7_0   pixel_7_1   pixel_7_2   pixel_7_3  \
count  1797.000000  1797.000000  1797.000000  1797.000000  1797.000000
mean      0.206455     0.000556     0.279354     5.557596    12.089037
std       0.984401     0.023590     0.934302     5.103019     4.374694
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000     0.000000     1.000000    11.000000
50%       0.000000     0.000000     0.000000     4.000000    13.000000
75%       0.000000     0.000000     0.000000    10.000000    16.000000
max      13.000000     1.000000     9.000000    16.000000    16.000000

       pixel_7_4   pixel_7_5   pixel_7_6   pixel_7_7       target
count  1797.000000  1797.000000  1797.000000  1797.000000  1797.000000
mean     11.809126     6.764051     2.067891     0.364496     4.490818
std       4.913947     5.900623     4.090548     1.860122     2.865304
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%      10.000000     0.000000     0.000000     0.000000     2.000000
50%      14.000000     6.000000     0.000000     0.000000     4.000000
75%      16.000000    12.000000     2.000000     0.000000     7.000000
max      16.000000    16.000000    16.000000    16.000000     9.000000

[8 rows x 65 columns]
```

```
df.head
   pixel_0_0  pixel_0_1  pixel_0_2  pixel_0_3  pixel_0_4  pixel_0_5  \
0        0.0        0.0        5.0       13.0        9.0        1.0
1        0.0        0.0        0.0       12.0       13.0        5.0
2        0.0        0.0        0.0        4.0       15.0       12.0
3        0.0        0.0        7.0       15.0       13.0        1.0
4        0.0        0.0        0.0        1.0       11.0        0.0

   pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  ...  pixel_6_7  pixel_7_0  \
0        0.0        0.0        0.0        0.0  ...        0.0        0.0
1        0.0        0.0        0.0        0.0  ...        0.0        0.0
2        0.0        0.0        0.0        0.0  ...        0.0        0.0
3        0.0        0.0        0.0        8.0  ...        0.0        0.0
4        0.0        0.0        0.0        0.0  ...        0.0        0.0

   pixel_7_1  pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  \
0        0.0        6.0       13.0       10.0        0.0        0.0
1        0.0        0.0       11.0       16.0       10.0        0.0
2        0.0        0.0        3.0       11.0       16.0        9.0
3        0.0        7.0       13.0       13.0        9.0        0.0
4        0.0        0.0        2.0       16.0        4.0        0.0

   pixel_7_7  target
0        0.0     0.0
1        0.0     1.0
2        0.0     2.0
3        0.0     3.0
4        0.0     4.0

[5 rows x 65 columns]
(1797, 64) (1797,)
```



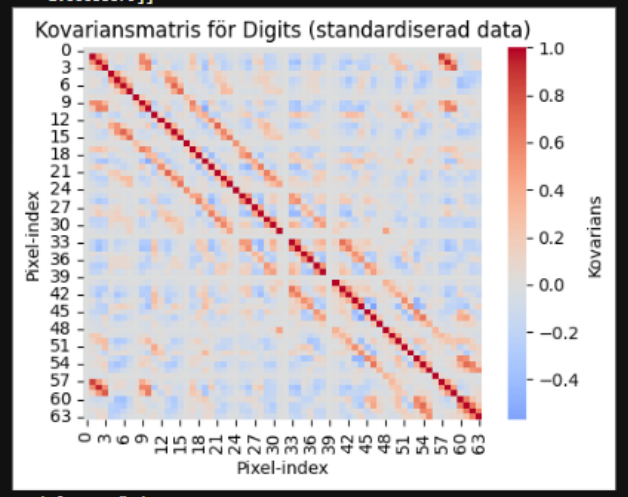

```
Siffra 0: 178
Siffra 1: 182
Siffra 2: 177
Siffra 3: 183
Siffra 4: 181
Siffra 5: 182
Siffra 6: 181
Siffra 7: 179
Siffra 8: 174
Siffra 9: 180
```

Datareduktion

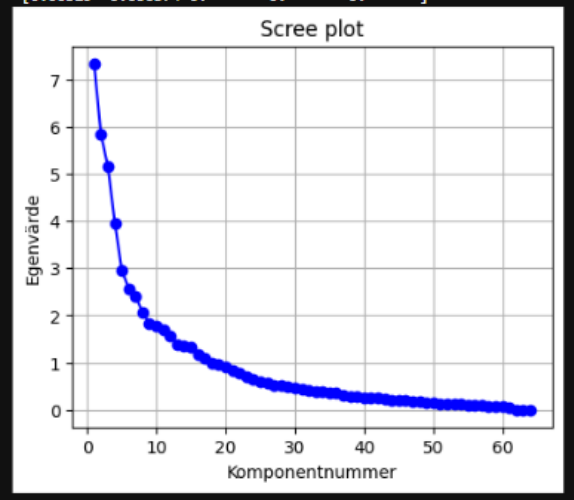
PCA

- Standardisera data (StandardScaler/MinMaxScaler etc)
- Beräkna kovariansmatris, fyndet var att variansen är utspridd över många huvudkomponenter.
- Hitta egenvärden och egenvektorer
- Projicera datan på egenvektorerna
- Scree plot

```python
# Kovariansmatris
cov_matrix = np.cov(x_scaled, rowvar=False)
print("Kovariansmatris från skalad data:")
print(cov_matrix)

# Heatmap, sanity check. Vi ser att varje pixel har perfekt korrelation med sig själv. Om diagonalen inte är röd så är något fel
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cov_matrix, cmap='coolwarm', center=0, ax=ax, cbar_kws={'label': 'Kovarians'})
ax.set_title("Kovariansmatris för Digits (standardiserad data)")
ax.set_xlabel("Pixel-index")
ax.set_ylabel("Pixel-index")
plt.tight_layout()
plt.show()

# Egenvärden och egenvektorer
# Hitta egenvärden och egenvektorer
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

# eigh returnerar i stigande ordning - vi vill ha fallande (störst först)
sorted_idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_idx]
eigenvectors = eigenvectors[:, sorted_idx]

print("Antal egenvärden:", len(eigenvalues))
print("Egenvektorer shape:", eigenvectors.shape)

# Värden per huvudkomponent
print("\nDe 10 största egenvärdena:")
print(eigenvalues[:10].round(3))
print("\nDe 5 minsta egenvärdena:")
print(eigenvalues[-5:].round(6))


# Screeplot
plt.figure(figsize=(5, 4))
plt.plot(range(1, len(eigenvalues)+1), eigenvalues, 'o-', color='b')
plt.title('Scree plot')
plt.xlabel('Komponentnummer')
plt.ylabel('Egenvärde')
plt.grid(True)
plt.show()
```




```
Antal egenvärden: 64
Egenvektorer shape: (64, 64)

De 10 största egenvärdena:
[7.345 5.835 5.154 3.966 2.966 2.572 2.406 2.069 1.83  1.79 ]

De 5 minsta egenvärdena:
[0.06329  0.050374 0.       0.       0.      ]
```

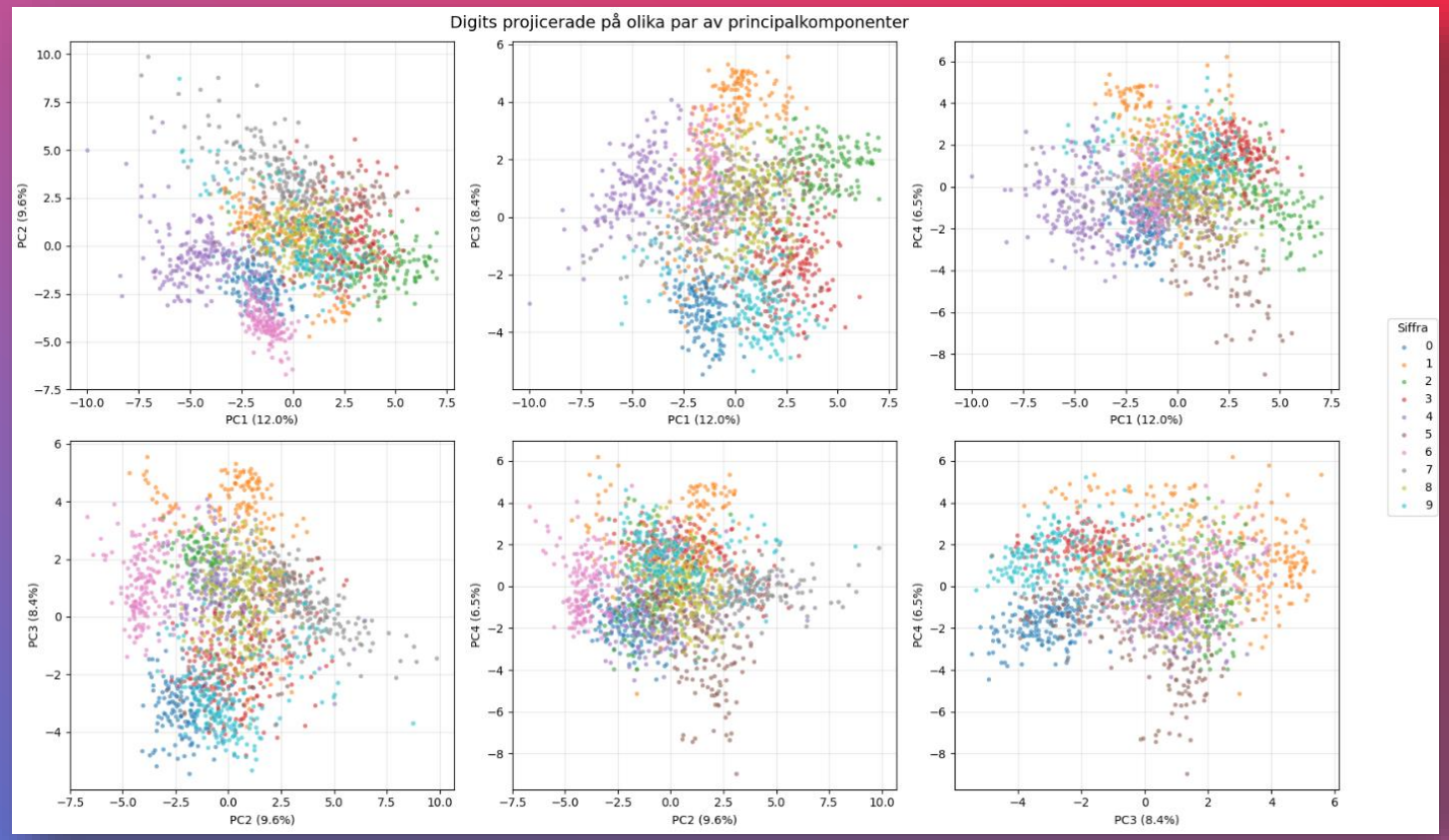


Datareduktion

PCA

- 2D-visualisering över två huvudkomponenter hjälper oss inte mycket att se tydliga grupperingar

```python
# Här kör vi en pairplot och plottar flera huvudkomponenter mot varandra

#Beräkna variance_ratio om den inte finns
variance_ratio = eigenvalues / eigenvalues.sum()

# Vilka par ska vi plotta?
pairs = [(0, 1), (0, 2), (0, 3),
         (1, 2), (1, 3), (2, 3)]

fig, axes = plt.subplots(2, 3, figsize=(16, 10))

for ax, (pc_x, pc_y) in zip(axes.flat, pairs):
    for i in range(10):
        mask = (y == i)
        ax.scatter(
            x_pca[mask, pc_x], x_pca[mask, pc_y],
            label=str(i), alpha=0.6, s=15,
            edgecolors='w', linewidth=0.2
        )
    ax.set_xlabel(f'PC{pc_x+1} ({variance_ratio[pc_x]:.1%})')
    ax.set_ylabel(f'PC{pc_y+1} ({variance_ratio[pc_y]:.1%})')
    ax.grid(True, alpha=0.3)

# En gemensam legend utanför plotten
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, title='Siffra',
           bbox_to_anchor=(1.02, 0.5), loc='center left')

plt.suptitle('Digits projicerade på olika par av principalkomponenter', fontsize=14)
plt.tight_layout()
plt.show()
```

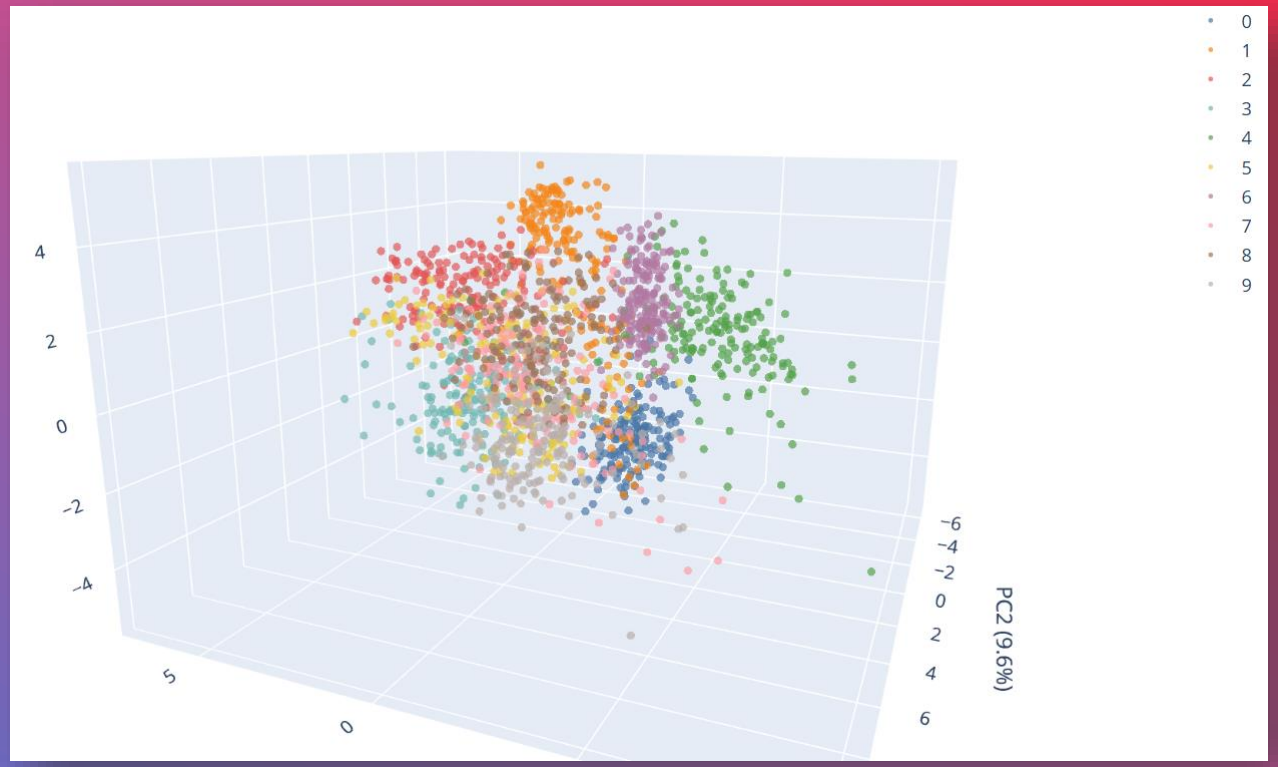


Datareduktion

PCA

- **3D-visualisering över tre huvudkomponenter tillför en del tydlighet**

```python
# Plotly för interaktiv karta, vi kan visualisera upp till tredje huvudkomponenten med en 3d-plot.


# Skapa en DataFrame för plotly
df_3d = pd.DataFrame({
    'PC1': x_pca[:, 0],
    'PC2': x_pca[:, 1],
    'PC3': x_pca[:, 2],
    'Siffra': y.astype(str)  # string så plotly behandlar det kategoriskt
})

fig = px.scatter_3d(
    df_3d, x='PC1', y='PC2', z='PC3',
    color='Siffra',
    color_discrete_sequence=px.colors.qualitative.T10,
    opacity=0.7,
    title=f'Digits i 3D-PCA ({variance_ratio[:3].sum():.1%} varians totalt)'
)

fig.update_traces(marker=dict(size=3))
fig.update_layout(
    scene=dict(
        xaxis_title=f'PC1 ({variance_ratio[0]:.1%})',
        yaxis_title=f'PC2 ({variance_ratio[1]:.1%})',
        zaxis_title=f'PC3 ({variance_ratio[2]:.1%})'
    ),
    width=900,
    height=700
)

fig.show(renderer="browser")
```

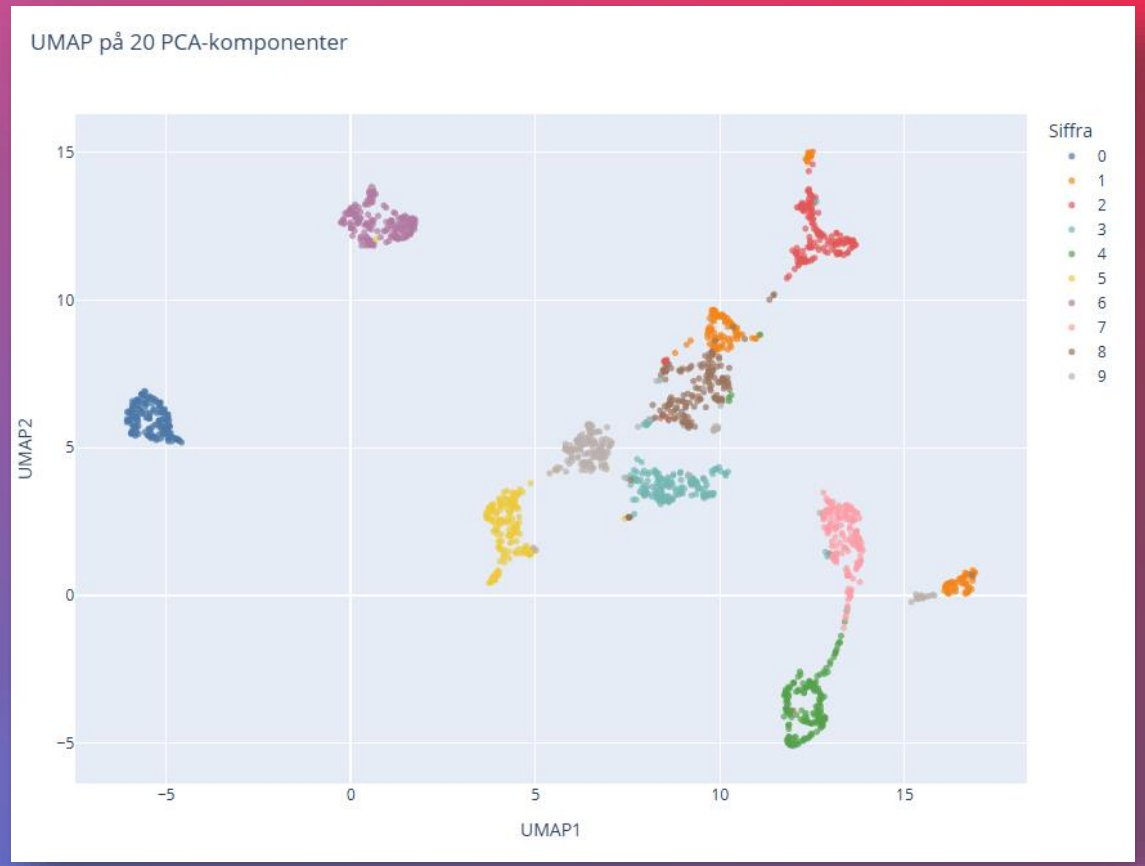


Datareduktion

UMAP

- Vid UMAP ser vi tydligare kluster, ett per siffra.
- 20 PCA-komponenter fångar en stor del av variansen!

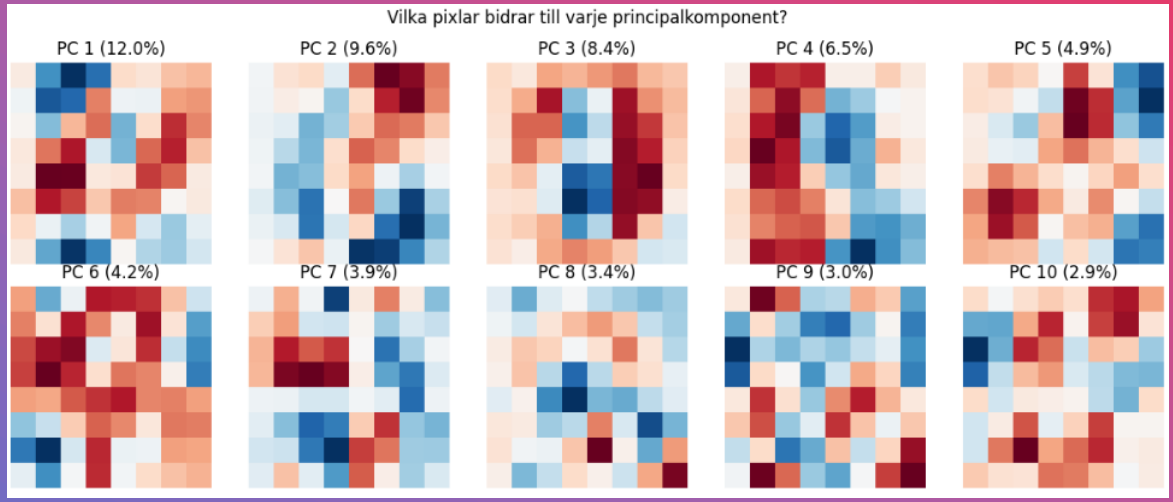
```python
# UMAP för att få med flera huvudkomponenter! Det är här vi verkligen börjar se tydliga grupperingar!

# Använd de första 20 PCA-komponenterna (behåller mycket info, tar bort brus)
x_pca_20 = x_pca[:, :20]
print(f"Använder {x_pca_20.shape[1]} komponenter som fångar {variance_ratio[:20].sum():.1%}")

# UMAP på 20 PCA-komponenter
reducer = UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
x_umap = reducer.fit_transform(x_pca_20)

df_umap = pd.DataFrame({
    'UMAP1': x_umap[:, 0],
    'UMAP2': x_umap[:, 1],
    'Siffra': y.astype(str)
})

fig = px.scatter(
    df_umap, x='UMAP1', y='UMAP2', color='Siffra',
    color_discrete_sequence=px.colors.qualitative.T10,
    opacity=0.7,
    title='UMAP på 20 PCA-komponenter'
)
fig.update_traces(marker=dict(size=5))
fig.update_layout(width=900, height=700)
fig.show(renderer="browser")
```



- UMAP på grunddatat

Datareduktion

EXTRA

- Undersökning av bidrag av olika variable
- Interaktiv visualisering
- Hyperparameter-justering I UMAP

```python
# Visualisering av principalkomponenterna som 8×8-bilder, svarar på vilka pixlar är viktigast för varje komponent.
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    component_image = eigenvectors[:, i].reshape(8, 8)
    ax.imshow(component_image, cmap='RdBu_r')
    ax.set_title(f'PC {i+1} ({variance_ratio[i]:.1%})')
    ax.axis('off')
plt.suptitle('Vilka pixlar bidrar till varje principalkomponent?')
plt.tight_layout()
plt.show()
```